Deliverable 12 — Working AI Prototype Evidence

TRAIL Scout: conversational discovery and trail synthesis

TRAIL | Member 4 — AI, Data & Finance

Project	Cartographic Pedestrian Exploration & Discovery Platform
Prepared for	BridgeAura Startup Challenge — final submission
Document basis	TRAIL repository handoff and Member 4 deliverable files

1. Prototype Summary

TRAIL Scout is documented as a working AI prototype integrated with the TRAIL frontend and backend. It offers two interaction modes: conversational discovery through /api/chat and structured trail generation through /api/generate-trail. The documented local frontend route is /ai-discovery, with the backend running on port 4000.

2. Architecture

```
User / Browser
    ↓
Next.js frontend
  • /ai-discovery
  • TrailChatbot
    ↓  POST /api/chat or /api/generate-trail
Express backend
  • routes + aiService.ts
  • curated POI dataset
    ↓
Gemini API (when configured)
    OR local keyword/vibe fallback
    ↓
Spot cards / structured trail dossier
```

3. How to Run Locally

Prerequisites: Node.js, Git, and (for Gemini mode) a Google AI Studio API key. Do not include the actual key in screenshots or submitted documents.

```
1. Open a terminal in the repository.
2. Backend:
   cd backend
   npm install
   Copy .env.example to .env and set GEMINI_API_KEY.
   npm run dev
3. In a second terminal, frontend:
   cd frontend
   npm install
   npm run dev
4. Open:
   http://localhost:3000/ai-discovery
5. Check backend:
   http://localhost:4000/health
6. Dataset endpoint:
   http://localhost:4000/api/spots
```

The repository's README/evidence file provides these local addresses. A localhost URL is not a public demo link; panel members cannot access it from their own devices unless the app is deployed or shown from the presenter's laptop.

4. Five Required Interaction Tests

#	Test prompt / action	Expected behaviour described in project evidence	Evidence status
1	"Find me a quiet morning coffee spot in Fontainhas"	Detect coffee + morning + quiet + Fontainhas; recommend matching curated POIs, e.g. Cafe Bodega.	Documented expected result; re-run live
2	"Plan me a 2-hour heritage walk with old architecture and colonial buildings"	Produce a heritage-focused sequence of stops and a duration-aware dossier.	Documented expected result; re-run live
3	Generate a 2.5-hour art, craft and culture trail in Panaji, max 4 km	Return structured trail JSON with stops, duration and metadata.	Documented expected result; re-run live
4	First ask for coffee spots; then say "I've already been to Cafe Bodega—suggest something different"	Exclude the named place and rank alternatives.	Documented expected result; re-run live
5	"What's good for an evening sunset experience in Panaji?"	Match evening/sunset intent to relevant viewpoint/waterfront options.	Documented expected result; re-run live

5. Results and What Must Be Captured

The repository's D12 evidence marks all five test interactions as passing, but its responses are described as expected/documented outputs and are not accompanied in the merged text by actual screenshots or a timestamped test log. For a defensible submission, run each test against the current code and capture: prompt, visible result, model/fallback source, timestamp, and any errors. Do not label an expected response as a newly verified live result.

Evidence checklist

- ■ Screenshot of /ai-discovery loaded
- ■ Screenshot/video of Test 1 output
- ■ Screenshot/video of Test 2 output
- ■ Screenshot of structured Test 3 trail
- ■ Screenshot showing Test 4 exclusion works
- ■ Screenshot of Test 5 sunset recommendations
- ■ Backend health check and dataset count
- ■ One fallback-mode test with the API key absent/unavailable
- ■ Redact API keys, private logs and personal information

6. Demo Script (60–90 seconds)

Opening: "This is TRAIL Scout, the discovery engine behind TRAIL. Instead of searching for one destination, a user describes the kind of walk they want."

Action: Enter: "I want a quiet morning coffee and heritage lanes in Fontainhas — about two hours." Run the synthesis and point to the generated stops, curator notes and duration.

Explain: "The system extracts preferences, matches them against our curated Panaji place dataset, and uses the AI layer to assemble a trail. If the external model is unavailable, the local matching fallback can still return recommendations from the dataset."

Close: "The next step is to expand the verified dataset and evaluate relevance with real users. These screens show the current prototype; our financial and market forecasts are separate planning assumptions."

7. Demo Readiness Checklist

- ■ Install dependencies before presentation day
- ■ Confirm both servers start without errors
- ■ Confirm GEMINI_API_KEY is configured securely (if using Gemini)

■ Test one query in Gemini mode and one in fallback mode
■ Keep a screen recording or screenshots as backup
■ Keep the browser at the AI Discovery page
■ Do not promise fixed latency or 100% uptime unless measured and qualified

8. Prototype Endpoints

Endpoint	Purpose
http://localhost:3000/ai-discovery	Trail synthesis UI
http://localhost:3000	Main frontend / chatbot entry point (as documented)
http://localhost:4000/health	Backend health status
http://localhost:4000/api/spots	Curated POI dataset endpoint
POST http://localhost:4000/api/chat	Conversational discovery
POST http://localhost:4000/api/generate-trail	Structured trail synthesis

Source basis: member4/D12_AI_PROTOTYPE_EVIDENCE.md and member4/D13_PITCH_CONTENT_AND_AUDIT.md. The evidence pack is a test plan and evidence checklist; capture actual screenshots/results from the running build before final submission.